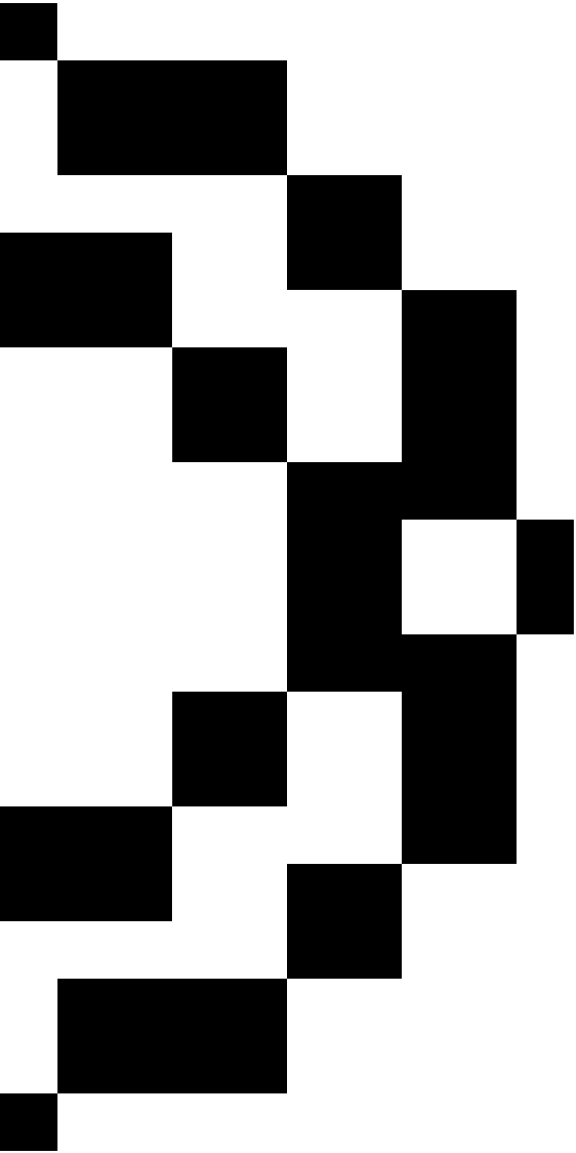




Oblique Decision Trees and Oblique Random Forests

Leonardo Vanneschi

lvanneschi@novaims.unl.pt

NOVA Information Management School (NOVA IMS)
Universidade Nova de Lisboa
Campus de Campolide
1070-312 Lisboa, Portugal

Oblique Decision Trees

A key challenge for decision trees is to identify the most appropriate split rule.

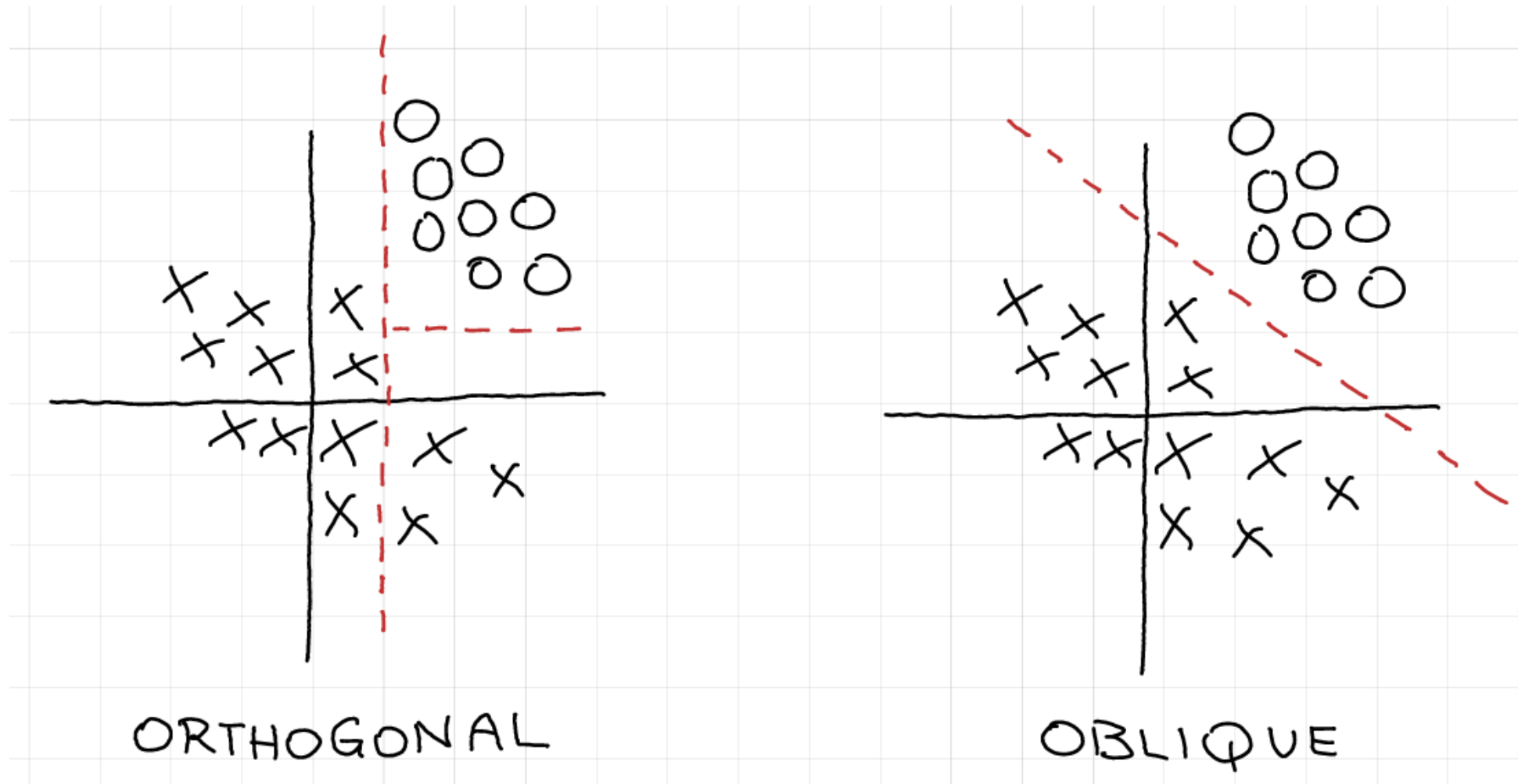
In the traditional approach (also called **Univariate Decision Trees**), splits are made using only *one variable*.

Geometrically, this allows us to perform only axis-parallel, or orthogonal, splits.

Oblique Decision Trees

Oblique Decision Trees, also called **Multivariate Decision Trees**, use multiple variables at each split, which allows us to model decision boundaries that are *oblique* to the attribute axis (see figure in the next slide).

Oblique Decision Trees



Oblique Decision Trees

Typically, oblique decision trees place combinations of the input features at the internal nodes and the form of those combinations is typically predetermined and fixed (even though extensions exist, where also the shape and the function at each node is optimized, for instance, by means of a method called Genetic Programming – studied in the optional discipline of the 4th trimester, called Neural and Evolutionary Learning).

One of the most popular choices, due to its simplicity, consists of positioning at each node of the Decision Tree a **linear combination** of the input features.

Oblique Decision Trees

Let $x_1, x_2, \dots, x_d$ be the input features, the idea is to position at each node of a Decision Tree a function like:

$$\sum_{i=1}^d w_i x_i + w_0 > 0$$

where $w_0, w_1, \dots, w_d$ are real-valued coefficients, to be optimized.

Oblique Decision Trees

For simplicity, in this presentation we will focus on the case where all the feature values $x_1, x_2, \dots, x_d$ are numeric.

Remark that categorical values can be converted into numeric data using several possible methods (e.g., the hot-one method).

While Multivariate Decision Trees is a more general term, that can potentially include also non-linear combinations of the variables, Oblique Decision Trees is usually intended as a more specific term, indicating the use of linear combinations of the variables, equivalent to hyperplanes that are oblique to the axes.

Oblique Decision Trees

If we observe the previous definition, Oblique Decision Trees are clearly a generalization of traditional Decision Trees, where all coefficients w_i , with $i = 1, 2, \dots, d$ **but one** are set to zero.

Oblique Decision Trees produce polygonal (polyhedral, in multiple dimensions) partitioning of the feature space, while traditional (orthogonal) Decision Trees produce hyper-rectangle partitioning, parallel to the feature axis.

Oblique Decision Trees

Problem: enumerating all possible splits and choose the best according to some criterion, as traditional Decision Trees do, is impractical/impossible for Oblique Decision Trees.

Furthermore, any non-exhaustive deterministic algorithm for searching through all these hyperplanes is prone to getting stuck in local minima.

Oblique Decision Trees

A common approach is to find locally “good” splits, but not spend excessive computational effort on improving the quality of those splits (while at the same time, possibly also try to keep the trees as small as possible in terms of number of nodes).

One of the simplest ways of doing this is by extending the well-known (orthogonal) CART algorithm of Decision Trees. This extension is called **CART with Linear Combination (CART-LC)**.

CART with Linear Combination

While CART looks for a split like:

$$x_j \leq t \quad \text{(using one variable at a time)}$$

CART-LC looks for a split like

$$w_1 x_1 + w_2 x_2 + \dots + w_d x_d \leq c$$

(a linear combination of the variables)

CART with Linear Combination

For each node in the tree, CART-LC makes the following steps:

1. Standardize the variables (so, no variable dominates just because of scale)
2. Initialize coefficients $w_1, w_2, \dots, w_d, c$ (see next slides for some hints)
3. Repeat until termination condition:
 - 3.1. For each variable x_i (with $i = 1, 2, \dots, d$):
 - 3.1.1. Try small changes to coefficient w_i
 - 3.1.2. Choose the change that most improve split quality
 - 3.2. Adjust threshold c
4. Remove unnecessary variables from the linear combination
5. Translate the split back to the original variable scale
6. Compare the best ordinary (orthogonal) CART split and keep the better one.

CART with Linear Combination

In practice, CART-LC implements a simple Hill Climbing algorithm to optimize the coefficients.

Hill Climbing is a Local Search method, that returns locally (and not necessarily globally) optimal solutions to optimization problems.

Remark: Hill Climbing and several more sophisticated optimization methods will be studied in depth in the discipline of the second semester Computational Intelligence for Optimization.

Let's now see some details of the CART-LC algorithm. After that, we will see a complete example showing the functioning of CART-LC.

Point 2: Initialization of the Coefficients

Even though there is no rule on how to initialize the coefficients, in practice CART-LC never uses a random initialization, because the search space (space of all possible coefficients values) is huge.

Instead, what is done in practice is to find the best orthogonal split $x_j \leq c$ with traditional CART and then initialize $w_j = 1$ and the other w_i , with $i \neq j$ as $w_i = 0$.

From there, we start the process of perturbing slightly the coefficients and see if there is an improvement in the quality of the split.

Point 2: Initialization of the Coefficients - Geometry

The previous process is equivalent to considering the best orthogonal (i.e., axis parallel) split and rotate it and slice it a little bit, locally, and check if this improves the quality of the split.

Note that this type of initialization makes point 6 of the algorithm redundant: each coefficient perturbation is retained only if the quality of the split is improved.

So, in the worst case scenario, if no improvement was found, we return the best ordinary (orthogonal) CART split.

How to Measure the Quality of a Split

Exactly like for ordinary CART, we want to choose the split that makes the two children of the tree “as pure as possible” (i.e., with as less mistakes as possible).

Measures to quantify impurity reduction:

- Gini Score
- Entropy Gain
- Variance Reduction (typically used for regression)
- etc...

Point 3.1.1. How to Slightly Change a Coefficient w_i

This is what we do to create a new candidate coefficient (and later we will test if this new coefficient improves the quality of the split or not).

The new candidate w_i is a slight modification of the current value of w_i

Point 3.1.1. How to Slightly Change a Coefficient w_i

Specifically, given a current split such as:

$$\sum_{j=1}^d w_j x_j \leq C$$

CART-LC explores, for various values of parameters δ and γ , the following split (for each variable x_i):

$$\sum_{j=1}^d w_j x_j - \delta \cdot (x_i + \gamma) \leq C$$

Point 3.1.1. How to Slightly Change a Coefficient w_i

In the previous equation, it is typical to try values for γ equal to -0.25 , 0 and 0.25 , and for δ (which tunes the importance of the step) it is typical to try several possible values (more on this later), always keeping the combination that maximizes the split quality.

In the next slides, we see how this perturbation modifies the inequality of the split.

Point 3.1.1. How to Slightly Change a Coefficient w_i

The inequality can be rewritten as:

$$w_1 x_1 + w_2 x_2 + \dots + w_i x_i - \delta x_i + \dots + w_d x_d \leq C + \delta \gamma$$
$$w_1 x_1 + w_2 x_2 + \dots + \underbrace{(w_i - \delta)}_{\substack{\uparrow \\ \text{PERTURBATION OF COEFFICIENT} \\ w_i}} x_i + \dots + w_d x_d \leq \underbrace{C + \delta \gamma}_{\substack{\uparrow \\ \text{PERTURBATION OF} \\ \text{THRESHOLD}}}$$

Notice that, in addition to what we wrote before, this is not only a perturbation of w_i , but also a perturbation of c .

Point 3.1.1. How to Slightly Change a Coefficient w_i

So, after all our attempts, with all values that we test for parameters δ and γ , let δ^* and γ^* be the optimal ones that we have found (the ones that maximize the split quality).

Before continuing the algorithm, we update w_i and c as:

$$w_i = w_i - \delta^* \quad \text{AND} \quad C = C + \delta^* \gamma^*$$

Warning: the article “A System for Induction of Oblique Decision Trees” by Murphy *et al.*, published on *Journal of Artificial Intelligence Research*, reports the math of the previous update in a wrong way (see the difference between the present document and the algorithm reported in Figure 4 in that article).

Point 3.1.1. How to Slightly Change a Coefficient w_i

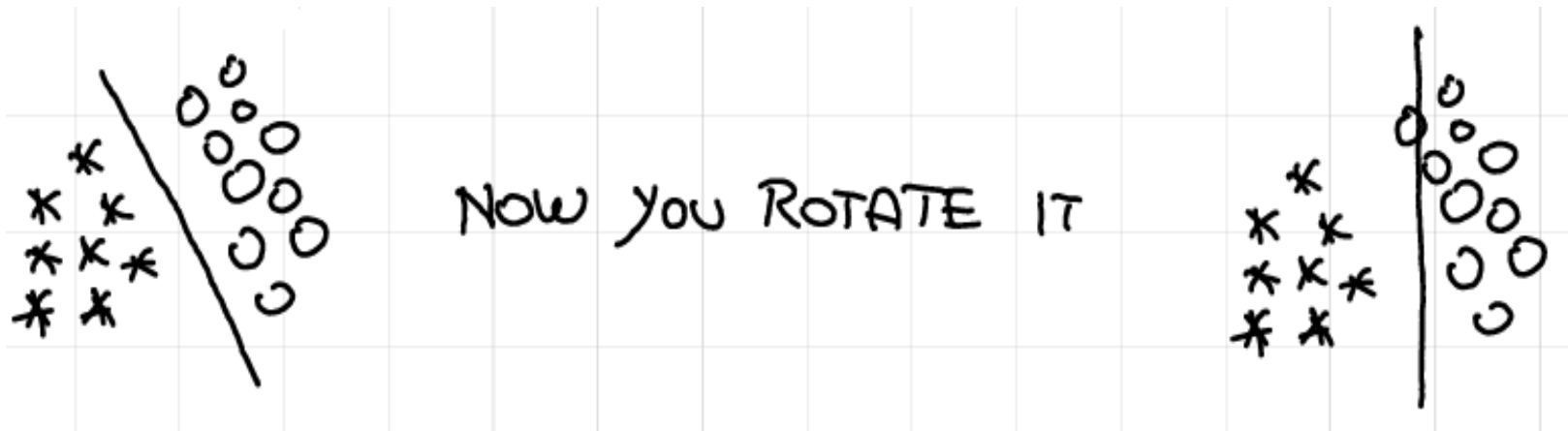
Notice that it makes a lot of sense, whenever we modify w_i to also modify c accordingly.

Modifying w_i is equivalent to rotate the hyperplane (a line in 2D). But after rotation, the best shift is no longer the same, and that's why it is appropriate to also modify c accordingly.

The figures in the next slide should help clarify this concept.

Point 3.1.1. How to Slightly Change a Coefficient w_i

Imagine you have a line partitioning data:



To make it still be a good separation, you need to accompany this rotation with a translation (i.e., a change of shift), see next slide.

Point 3.1.1. How to Slightly Change a Coefficient w_i

If you consider the line after rotation in the figure of the previous slide and you translate it, you can obtain the following situation:



... and this is a better split than the situation before the translation.

Point 3.2. Adjust Threshold c

When all coefficients have been adjusted one by one (and threshold c accordingly at each step, as we have just seen!), CART-LC still makes another, final, adjustment to threshold c .

This is usually done by testing several values slightly smaller and slightly bigger than the current c and considering as the final value the one that maximizes the split quality.

Point 4. Remove Unnecessary Variables from the Linear Combination

We may want to remove from the linear combination the variables that have a coefficient “close to zero”.

This makes the tree simpler, easier to interpret and less noisy.

However, for each variable we want to eliminate, it is always a good idea to test how this elimination impacts the split quality and make this elimination only if the impact is “marginal”.

Termination Condition

The modification of all the coefficients, one by one, is repeated for several iterations, until a termination condition is reached.

The CART-LC algorithm terminates when two consecutive updates (of all coefficients) do not improve the split quality *significantly* (say, when the improvement is smaller than a given, predefined, ε), **OR** after that a predefined maximum number of iterations has been performed.

Example of Functioning of CART-LC

Consider the following toy binary classification dataset, where x_1 and x_2 are input variables:

OBSERVATION ID		x_1	x_2	y
	B1	1	5	BLUE
	B2	2	4	BLUE
	B3	3	3	BLUE
	B4	7	1	BLUE
	B5	8	1	BLUE
	R1	2	6	RED
	R2	3	5	RED
	R3	4	4	RED
	R4	5	3	RED
	R5	6	2	RED

Example of Functioning of CART-LC

Notice that the dataset is completely balanced, so the impurity at the root is maximum.

In other words, a classifier characterized by just one leaf that contains all observations is very bad.

Definition of GINI Score

The **GINI Score of a feature x** (also called **weighted impurity of x**) is:

$$\text{GINI_SCORE}(X) = \sum_{k=1}^N w_k \cdot \text{GINI_IMPURITY}_k$$

where:

- N is the total number of possible values that x can take
- For each $k = 1, 2, \dots, N$: w_k is the probability that x takes the k^{th} possible value.

(... continues in the next slide with the definition of GINI_IMPURITY_k).

Definition of GINI Impurity

Also, for each $k = 1, 2, \dots, N$:

$$\text{GINI_IMPURITY}_k = 1 - \sum_{j=1}^M p_{jk}^2$$

where:

- M is the total number of possible values that the target can take
- for all $j = 1, 2, \dots, M$: p_{jk} is the probability that x takes its k^{th} value AND that the target takes its j^{th} possible value.

The next examples should clarify this.

The GINI Impurity of the Random Classifier

A classifier characterized by just one leaf that contains all observations would have the following GINI Impurity:

$$\text{GINI}(\text{Root}) = 1 - \underbrace{\left(\frac{5}{10}\right)^2}_{\substack{\text{FOR} \\ \text{CLASS} \\ \text{BLUE}}} - \underbrace{\left(\frac{5}{10}\right)^2}_{\substack{\text{FOR} \\ \text{CLASS} \\ \text{RED}}} = 0.5$$

In the degenerate case of a tree composed by only one node, we define the GINI Score identical to the GINI Impurity, so this is also the GINI Score.

Example of Functioning of CART-LC

In this example, we will see that:

- An ordinary axis-parallel split is not very good at the root.
- An oblique split at the root is much better;
- Then, one final axis-parallel split finishes the job (of removing all impurities).

Example of Functioning of CART-LC

In this example, both variables are already in the same scale, so to keep math simpler we will skip the initial step of data normalization/standardization.

Given that the initialization will be given by the best axis-parallel split, let's start by finding that split.

Example of Functioning of CART-LC

Let's test the following split: $x_2 \leq 1.5$

We have that the left child of that split (i.e., the observations for which the condition is true) will contain observations B4 and B5 (2 BLUE, 0 RED).

The right child, instead, contains observations B1, B2, B3, R1, R2, ..., R5 (3 BLUE and 5 RED)

OBSERVATION ID	x_1	x_2	y
B1	1	5	BLUE
B2	2	4	BLUE
B3	3	3	BLUE
B4	7	1	BLUE
B5	8	1	BLUE
R1	2	6	RED
R2	3	5	RED
R3	4	4	RED
R4	5	3	RED
R5	6	2	RED

Example of Functioning of CART-LC

In such a situation, concerning the impurity of the left branch, we observe that the left branch is pure BLUE, which allows us to conclude:
 $\text{GINI_IMPURITY(LEFT)} = 1 - 1^2 - 0^2 = 0$

Concerning the impurity of the right branch, we have:

$$\text{GINI_IMPURITY(RIGHT)} = 1 - \left(\frac{3}{8}\right)^2 - \left(\frac{5}{8}\right)^2 \approx 0.47$$

OBSERVATION ID	x_1	x_2	y
B1	1	5	BLUE
B2	2	4	BLUE
B3	3	3	BLUE
B4	7	1	BLUE
B5	8	1	BLUE
R1	2	6	RED
R2	3	5	RED
R3	4	4	RED
R4	5	3	RED
R5	6	2	RED

Example of Functioning of CART-LC

So, the GINI Score (or weighted impurity) after the split is:

$$\frac{2}{10} \cdot 0 + \frac{8}{10} \cdot 0,47 \approx 0,375$$

Which allows us to conclude that **this split gives us the following Gain:**

$$\text{GAIN} = 0,5 - 0,375 = 0,125$$

OBSERVATION ID	x_1	x_2	y
B1	1	5	BLUE
B2	2	4	BLUE
B3	3	3	BLUE
B4	7	1	BLUE
B5	8	1	BLUE
R1	2	6	RED
R2	3	5	RED
R3	4	4	RED
R4	5	3	RED
R5	6	2	RED

Gain

In the previous slide, the term Gain refers to the difference between the previous GINI Score and the current one.

Notice: the smaller the GINI Score, the bigger the Gain!

Example of Functioning of CART-LC

Note: this gives us the same gain as $x_1 \leq 6.5$ and there are no single variable splits that can give a larger gain.

So, the best axis-parallel split gives a gain of **0.125**.

That is not terrible, but not great either...

OBSERVATION ID	x_1	x_2	y
B1	1	5	BLUE
B2	2	4	BLUE
B3	3	3	BLUE
B4	7	1	BLUE
B5	8	1	BLUE
R1	2	6	RED
R2	3	5	RED
R3	4	4	RED
R4	5	3	RED
R5	6	2	RED

Example of Functioning of CART-LC

Let us choose, for instance, $x_1 \leq 6.5$ as the result of the initialization.

In linear combination form, it is:

$$1 \cdot x_1 + 0 \cdot x_2 \leq 6.5$$

So, $w_1 = 1$, $w_2 = 0$ and $c = 6.5$.

Let's first keep $w_1 = 1$ and let's perturb w_2
(see next slide)

OBSERVATION ID	x_1	x_2	y
B1	1	5	BLUE
B2	2	4	BLUE
B3	3	3	BLUE
B4	7	1	BLUE
B5	8	1	BLUE
R1	2	6	RED
R2	3	5	RED
R3	4	4	RED
R4	5	3	RED
R5	6	2	RED

Example of Functioning of CART-LC

Let's first keep $w_1 = 1$ and let's perturb w_2

Let's start with $w_2 = 0.25$ (so: $\delta = -0.25$).

Let's put the output of the new linear combination in a table (see the next slide)

OBSERVATION ID	x_1	x_2	y
B1	1	5	BLUE
B2	2	4	BLUE
B3	3	3	BLUE
B4	7	1	BLUE
B5	8	1	BLUE
R1	2	6	RED
R2	3	5	RED
R3	4	4	RED
R4	5	3	RED
R5	6	2	RED

Example of Functioning of CART-LC

OBSERVATION ID	x_1	x_2	$x_1 + 0.25x_2$	y
B1	1	5	2.25	BLUE
B2	2	4	3	BLUE
B3	3	3	3.75	BLUE
B4	7	1	7.25	BLUE
B5	8	1	8.25	BLUE
R1	2	6	3.50	RED
R2	3	5	4.25	RED
R3	4	4	5	RED
R4	5	3	5.75	RED
R5	6	2	6.50	RED

Example of Functioning of CART-LC

Let's consider $c = 3.25$, then we have:

- Left child: B1, B2 (Pure BLUE)
- Right child: 3 BLUE, 5 RED

This gives exactly the same weighted impurity (and consequently the same gain) as before:

$$\text{GINI}_{\text{SPLIT}} = 0.375, \text{ GAIN} = 0.125$$

So, this linear combination does not improve the split quality.

Example of Functioning of CART-LC

Now, let $w_2 = 0.5$. In this case, we have:

OBSERVATION ID	x_1	x_2	$x_1 + 0,5 x_2$	y
B1	1	5	3.5	BLUE
B2	2	4	4	BLUE
B3	3	3	4.5	BLUE
B4	7	1	7.5	BLUE
B5	8	1	8.5	BLUE
R1	2	6	5	RED
R2	3	5	5.5	RED
R3	4	4	6	RED
R4	5	3	6.5	RED
R5	6	2	7	RED

Example of Functioning of CART-LC

Let's now consider $c = 4.75$. We have:

- Left child: B1, B2, B3 (PURE BLUE)
- Right child: 2 BLUE, 5 RED

So:

$$\text{LEFT IMPURITY : } \text{GINI}(L) = 0$$

$$\text{RIGHT IMPURITY : } \text{GINI}(R) = 1 - \left(\frac{2}{7}\right)^2 - \left(\frac{5}{7}\right)^2 \simeq 0.4082$$

$$\text{GINI}_{\text{SPLIT}} = \frac{3}{10} \cdot 0 + \frac{7}{10} \cdot 0.4082 \simeq 0.2857$$

$$\text{GAIN} = 0.5 - 0.2857 = 0.2143$$

Example of Functioning of CART-LC

We have obtained a gain of **0.2143**, which is better than the previous one (that was **0.125**).

So, changing w_2 was beneficial.

Example of Functioning of CART-LC

Now, let $w_2 = 1$. We have:

OBSERVATION ID	x_1	x_2	$x_1 + x_2$	y
B1	1	5	6	BLUE
B2	2	4	6	BLUE
B3	3	3	6	BLUE
B4	7	1	8	BLUE
B5	8	1	9	BLUE
R1	2	6	8	RED
R2	3	5	8	RED
R3	4	4	8	RED
R4	5	3	8	RED
R5	6	2	8	RED

Example of Functioning of CART-LC

Now, let us consider $c = 7$. We have:

- Left child: B1, B2, B3 (pure BLUE)
- Right child: 2 BLUE, 5 RED

This gives the same impurity as the previous good split:

$$GAIN = 0.2143$$

So, $w_2 = 1$ is equally good.

Example of Functioning of CART-LC

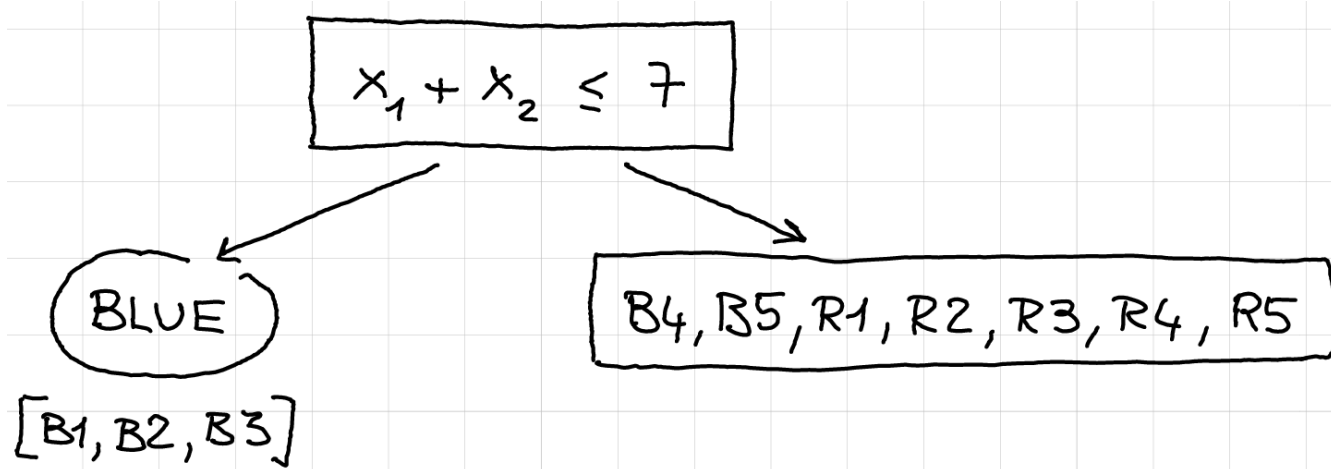
In this example, we keep $w_2 = 1$ just because the linear combination is simpler to read.

So, our split becomes:

$$x_1 + x_2 \leq 7$$

Example of Functioning of CART-LC

So, we now have a preliminary tree like:



Given that the right leaf has mixed BLUE and RED, and assuming that the maximum allowed tree depth has not been reached yet, we split again that leaf.

Example of Functioning of CART-LC

So, we now work only with this part of the training set (the part composed by only the observations that belong to the right branch):

OBSERVATION ID	x_1	x_2	y
B4	7	1	BLUE
B5	8	1	BLUE
R1	2	6	RED
R2	3	5	RED
R3	4	4	RED
R4	5	3	RED
R5	6	2	RED

Example of Functioning of CART-LC

The impurity of the right leaf is:

$$\text{GINI_IMPURITY} = 1 - \left(\frac{2}{7}\right)^2 - \left(\frac{5}{7}\right)^2 \simeq 0.4082$$

Let us now consider the best axis-parallel split:

$$x_2 \leq 1.5$$

Example of Functioning of CART-LC

With this split, we have:

- Left child: B4, B5 (pure BLUE)
- Right child: R1, R2, R3, R4, R5 (pure RED)

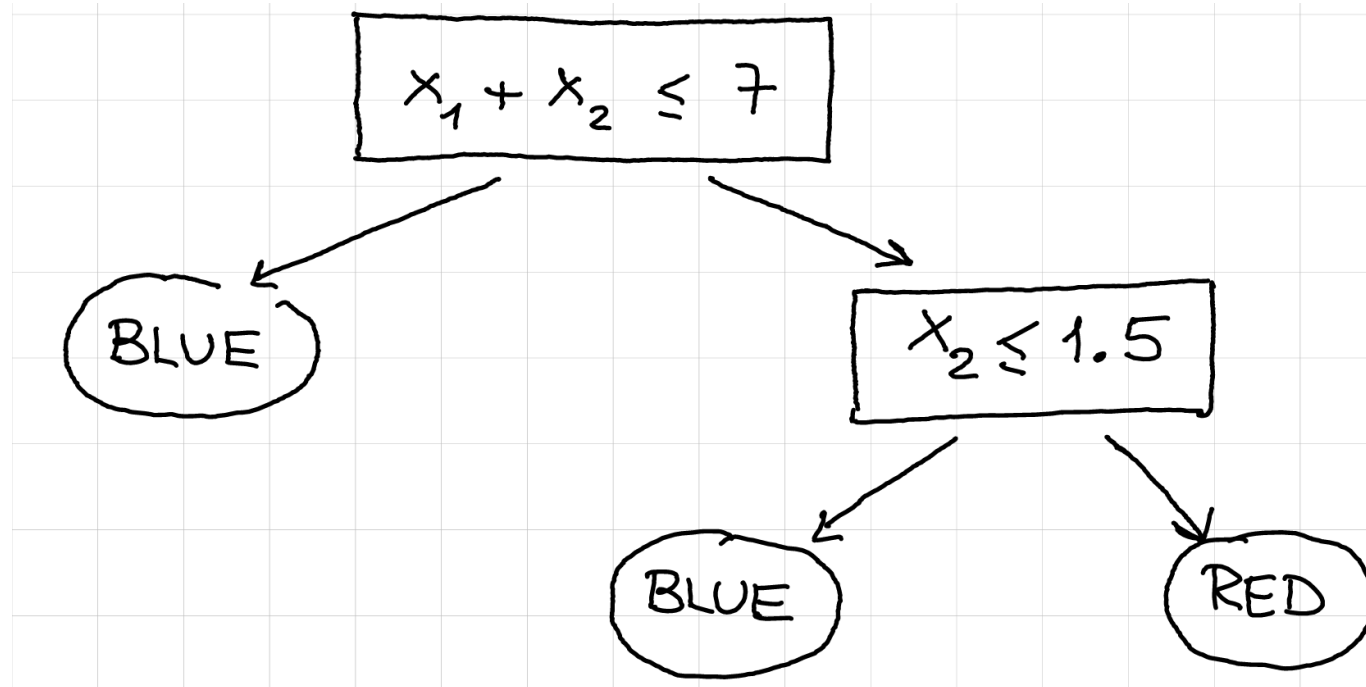
Therefore:

$$\text{GINI}_{\text{SPLIT}} = 0$$

which indicates that this split is perfect. There is no need to search for a more complex, oblique, split here.

Example of Functioning of CART-LC

So, we have the following final tree, that makes a perfect job in classifying our data:



Example of Functioning of CART-LC - Discussion

A traditional (orthogonal) CART tree would usually need more “staircase-like” splits to isolate the diagonal BLUE group (1, 5), (2, 4), (3, 3) and the bottom-right BLUE group (7, 1), (8, 1).

CART-LC, instead, is able to compress much of that structure into one diagonal split at the root.

This is exactly the conceptual advantage of Oblique Trees.

(end of the example)

Possible Alternatives to CART-LC

Besides CART-LC, several other algorithms have been defined to develop Oblique Trees.

Basically, these algorithms differ between each other for the method they employ to determine the coefficients $w_1, w_2, \dots, w_d$ and the threshold c .

One popular alternative to CART-LC is an algorithm called OC1, that uses *Simulated Annealing* to find those values.

Simulated Annealing is (another) popular optimization algorithm that will be studied in detail in the discipline of Computational Intelligence for Optimization, that will take place in the second semester.

Oblique Random Forests

Independently from the algorithm used to create oblique trees, several of these trees can be combined to form a forest.

As for traditional Random Forests, every time we want to find a new split in a tree, we create a bootstrapped dataset, and we select a random subset of the variables.

Besides this, typically, in Oblique Random Forests, at each tree node, the optimization of parameters $w_1, w_2, \dots, w_d$ (where d , this time, is the number of variables we have chosen) is made including also a regularization term, for instance like in Ridge Regression.

Oblique Random Forests

In other words, each split is obtained by minimizing an objective of the form:

$$J(\mathbf{w}, b) = \mathcal{L}_{\text{split}}(\mathbf{w}, b) + \lambda \Omega(\mathbf{w})$$

where:

- $\mathbf{w}$ is the vector of coefficients defining the oblique split,
- b is the threshold (bias),
- $\mathcal{L}_{\text{split}}$ measures the quality of the split,
- $\Omega(\mathbf{w})$ is a regularization term,
- λ controls the trade-off between split quality and regularization.

Oblique Random Forests

As in the previous example, the split quality is usually based on the impurity of the two child nodes. For example:

$$\mathcal{L}_{\text{split}} = \frac{N_L}{N} I(L) + \frac{N_R}{N} I(R)$$

where:

- N_L and N_R are the numbers of samples sent to the left and right children,
- N is the total number of samples,
- $I(\cdot)$ is an impurity measure such as:
 - Gini impurity,
 - entropy,
 - variance (for regression),
 - etc...

Oblique Random Forests

Using the previous definition of impurity, it is typical, as in the previous example, to maximize the impurity reduction (information gain)

$$\Delta I = I(\text{parent}) - \frac{N_L}{N} I(L) - \frac{N_R}{N} I(R)$$

which is mathematically equivalent to minimizing the weighted child impurity.

Oblique Random Forests

Concerning the regularization term, the most common choices are:

L1 regularization (sparse oblique trees)

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_i |w_i|$$

which encourages many coefficients to become exactly zero, producing sparse and more interpretable splits.

Or... (it continues in the next slide)

Oblique Random Forests

... Another possible alternative is:

L2 regularization

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_i w_i^2$$

which discourages excessively large coefficients and improves generalization.

Oblique Random Forests

A typical optimization problem is therefore:

$$\min_{\mathbf{w}, b} \left(\frac{N_L}{N} I(L) + \frac{N_R}{N} I(R) + \lambda \|\mathbf{w}\|_1 \right)$$

or, equivalently:

$$\max_{\mathbf{w}, b} (\Delta I - \lambda \|\mathbf{w}\|_1).$$

These are the formulations most commonly encountered in Oblique Random Forests.

Oblique Random Forests

Parameter λ is the regularization parameter. If $\lambda = 0$, then all focus is on model's accuracy. Otherwise, the larger λ , the more focus on regularization.

In practical terms, it is typical to test various values of $\lambda = 10^i$, with $i = \{-5, -4, \dots, 5\}$ and choosing the λ that allows to obtain the best **out-of-bag** result for that tree node.

Remark: regularization exists to improve generalization, so we definitely need out-of-bag samples to test it.

Oblique Random Forests

Usually, Oblique Random Forests have two *main* parameters:

- d , which is the number of random features selected for implementing the binary split at each tree node
- the number of trees in the forest (ensemble size).

Oblique Random Forests – Generalization Phase

As usual in Random Forests, predictions on unseen samples are obtained by running those samples down each one of the trees and then pooling the decision, by joining together all the single results (voting for classification and mean or median for regression).